Blockchain - NEP Sem VI

NMCPC62 : Cryptocurrency and Blockchain Development

Module - 5: Building Decentralized Applications (DApps)

(7 Hours)

Instructors : Geocey Shejy, Lifna C S, Pradnya Raut

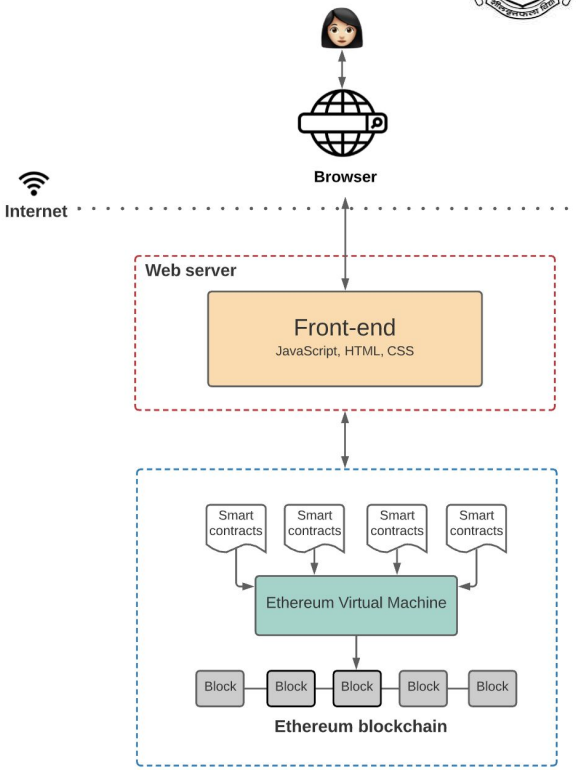
Department of Computer Engineering, VESIT, Mumbai

Topics to be covered

1. Architecture of Decentralized Applications,
2. Interaction Between Smart Contracts and Frontend Using Web3.js,
3. Tools for DApp Development: Web3.js, Ethers.js, and Metamask
4. Building and Deploying a Simple DApp on Ethereum Testnet,
5. Integrating Cryptocurrency Payments in DApps Using Payment Gateways,
6. Performance Optimization and Scaling Solutions for DApps,
7. Testing and Deploying DApps to Production Environments

1. Architecture of Decentralized Applications

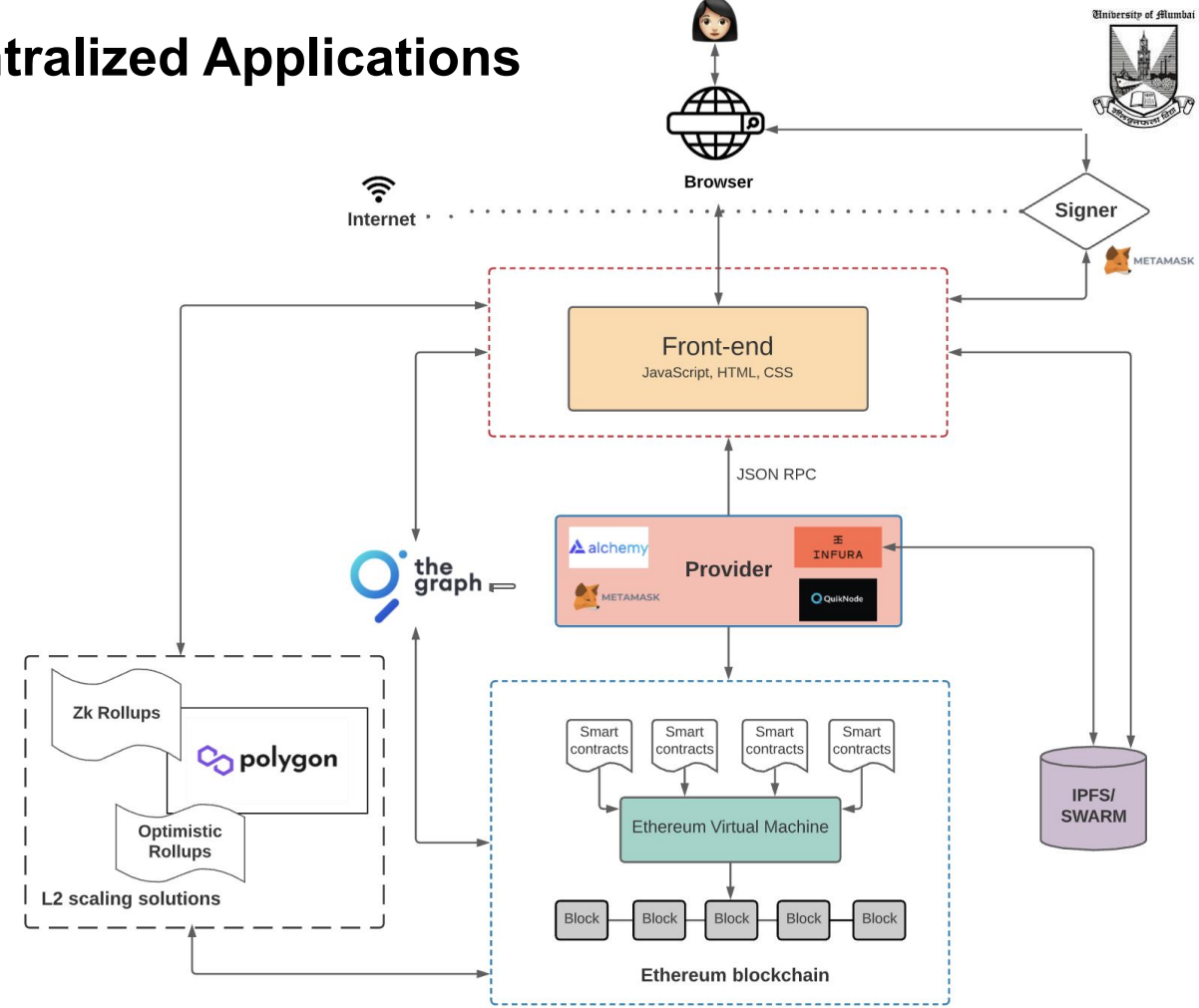
1. **Ethereum blockchain:**
   - Securely executes and verifies application code, called smart contracts.
   - DApps use the Ethereum blockchain for data storage.
2. **Smart Contracts:**
   - DApps use smart contracts to define the state changes happening on the blockchain.
   - A smart contract is a collection of code and data that resides at a specific address on the Ethereum Blockchain and runs on the Ethereum blockchain.
3. **Ethereum Virtual Machine(EVM):**
   - Global virtual computer that executes the logic defined in the smart contracts and processes the state changes that happen on this Ethereum network.
4. **Front-end:**
   - The part of the DApps, that users can see and interact with such as the graphical user interface(GUI),
   - Communicates with the application logic defined in smart contracts.



1. Architecture of Decentralized Applications

Addressing

- Storage,
- Querying
- Scalability



Note :

Detailed Overview of DApp is covered in Module 2

(Please refer M2-PPT - slide No. 195-203)

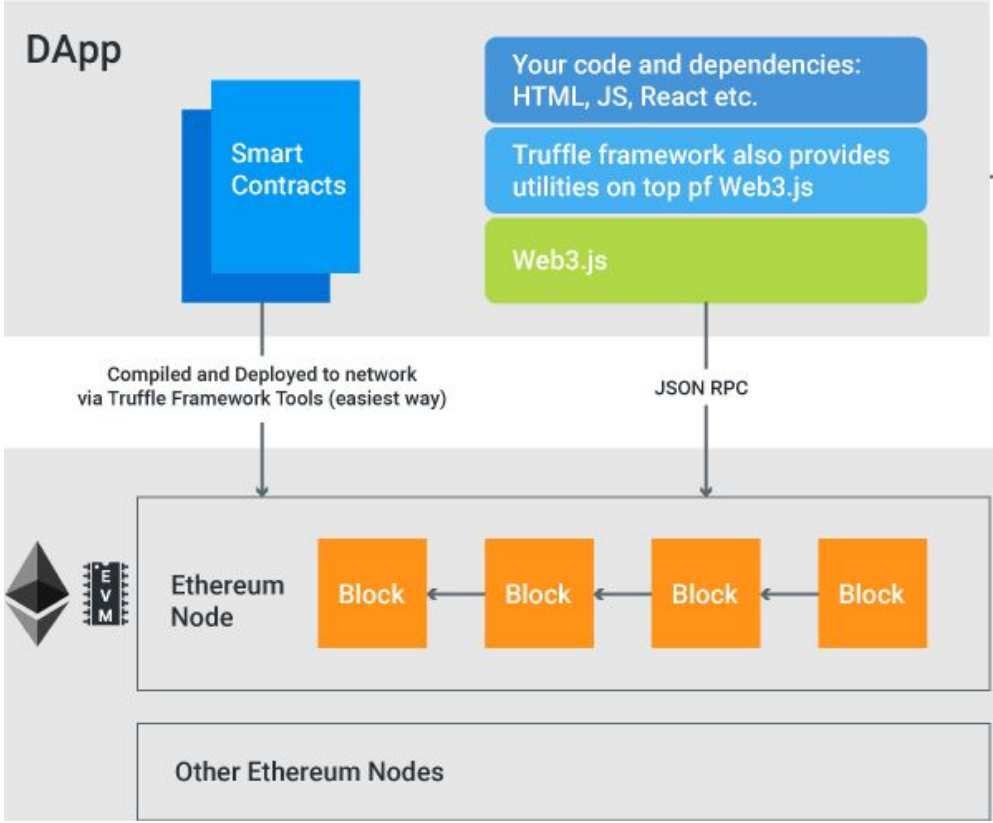
2. Interaction Between Smart Contracts and Frontend Using Web3.js

Typical DApp architecture: Frontend (React/HTML/JS) → Web3.js → Ethereum Node → Smart Contract

- Frontend : sends requests using Web3.js
- Web3.js : communicates with the blockchain node
- smart contract : executes logic and returns results

Web3.js

bridge between frontend and blockchain



2. Interaction Between Smart Contracts and Frontend Using Web3.js

Step - 1 : Write & Compile Smart Contract

- Write contract in Solidity
- Compile it to generate:
  - ABI (Application Binary Interface) : defines how frontend interacts with contract functions
  - Bytecode

Step - 2 : Deploy Smart Contract

- Use tools like: Hardhat / Truffle / Remix
- After deployment, we get: Contract Address

Step - 3. Setup Frontend Environment

- Install Web3.js: **npm install web3**
- Import in frontend: **import Web3 from "web3";**

2. Interaction Between Smart Contracts and Frontend Using Web3.js

Step - 4 : Connect to Blockchain (Provider)

- Method A: Local Node (Hardhat/Ganache) ⇒ `const web3 = new Web3("http://127.0.0.1:8545");`
- Method B: Browser Wallet (MetaMask) ⇒

```
const web3 = new Web3(window.ethereum);
await window.ethereum.enable();
```

Step - 5 : Create Contract Instance in Frontend

```
const contract = new web3.eth.Contract(ABI, contractAddress);
```

- Uses: ABI (interface) & Address (location of contract)

Step - 6 : Read Data from Smart Contract (Call) : `const value = await contract.methods.myFunction().call();`

- No gas fee & **Does NOT** change blockchain state
- Ex : `const myNumber = await contract.methods.myNumber().call();`

2. Interaction Between Smart Contracts and Frontend Using Web3.js

Step - 7. Write Data to Smart Contract (Send Transaction)

```
const accounts = await web3.eth.getAccounts();

await contract.methods.setValue(10).send({
  from: accounts[0]
});
```

- Requires gas & Updates blockchain state
- Ex: updating a variable triggers a transaction and returns a receipt

Step - 8. Handle Events (Optional)

```
contract.events.MyEvent()
.on('data', event => {
   console.log(event);
});
```

- Used for: UI updates & Notifications

3. Tools for DApp Development: Web3.js

- **JavaScript library** used to interact with Ethereum blockchain from the frontend.
- Key Features
  - Connects frontend to blockchain nodes
  - Interacts with smart contracts using ABI
  - Supports:
    - Accounts management
    - Transactions
    - Events handling
- Common Uses
  - Reading smart contract data (.call())
  - Sending transactions (.send())
  - Listening to contract events
- Ex.

```
const web3 = new Web3(window.ethereum);
const contract = new web3.eth.Contract(ABI, address);
```

- Advantages
  - Widely used and well-documented
  - Supports multiple Ethereum providers
- Limitation : Slightly heavier and more complex than newer libraries

3. Tools for DApp Development: Ethers.js

- **Lightweight JavaScript library** for interacting with Ethereum, often preferred in modern DApps.
- Key Features
  - Simpler and cleaner API than Web3.js
  - Better handling of Wallets, Signers, Transactions
- Common Uses
  - Contract interaction
  - Wallet integration
  - Signing transactions
- Ex:

```
import { ethers } from "ethers";

const provider = new ethers.providers.Web3Provider(window.ethereum);
const signer = provider.getSigner();
const contract = new ethers.Contract(address, ABI, signer);
```

- Advantages
  - Lightweight and faster
  - Cleaner syntax
  - Better TypeScript support
- Limitation
  - Smaller ecosystem compared to Web3.js (but growing fast)

3. Tools for DApp Development: Metamask

METAMASK

- A **browser extension + mobile app wallet** used to connect users to blockchain applications.
- Key Features
  - Stores Ethereum accounts securely
  - Signs transactions
  - Acts as a bridge between browser and blockchain
- Role in DApp
  - Provides **window.ethereum** object
  - Prompts user for: Transaction approval & Account access
- Example Flow
  1. User connects wallet
  2. MetaMask provides account
  3. User approves transaction
  4. Transaction sent to blockchain
- Advantages
  - Easy to use
  - Widely supported
  - Secure key management
- Limitation :
  - Requires user interaction (manual approval)

3. Tools for DApp Development - Comparison

Feature	Web3.js	Ethers.js	MetaMask
Type	JS Library	JS Library	Wallet
Purpose	Blockchain interaction	Blockchain interaction	User authentication
Complexity	Moderate	Simple	Very easy
Performance	Moderate	Faster	Depends on user
Role	API layer	API layer	Wallet + Provider

Courtesy : Web3.js - Docs

4. Building and Deploying a Simple DApp on Ethereum Testnet

1. Prerequisites

Before starting, ensure you have:

- Basic knowledge of Solidity
- Installed:
  - Node.js
  - MetaMask
  - Remix IDE OR Visual Studio Code
- Test ETH from a faucet (e.g., Sepolia faucet)

2. Create a Simple Smart Contract

Example: Counter Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Counter {
    uint public count;
    function increment() public {
        count += 1;
    }
    function decrement() public {
        count -= 1;
    }
}
```

4. Building and Deploying a Simple DApp on Ethereum Testnet

3. Compile the Smart Contract

Using Remix IDE:

1. Open Remix
2. Create a new file `Counter.sol`
3. Paste the code
4. Go to Solidity Compiler tab
5. Click Compile

4. Deploy Contract to Ethereum Testnet

1. Go to Deploy & Run Transactions tab
2. Select Environment:

   - Injected Provider - MetaMask
3. Connect your MetaMask wallet
4. Choose Testnet (e.g., Sepolia)
5. Click Deploy
6. Confirm transaction in MetaMask

4. Building and Deploying a Simple DApp on Ethereum Testnet

5. Get Contract Address & ABI

After deployment:

- Copy Contract Address
- Copy ABI (from compilation details)

These are required to connect frontend → blockchain.

6. Set Up Frontend (Basic DApp)

Create a simple frontend using HTML + JavaScript

Install dependencies:

```
npm init -y
npm install web3
```

7. Connect Frontend to Smart Contract

Example using Web3.js:

```
const web3 = new Web3(window.ethereum);
const contractAddress = "YOUR_CONTRACT_ADDRESS";
const abi = [/* ABI HERE */];
const contract = new web3.eth.Contract(abi,
contractAddress);
async function increment() {
   const accounts = await web3.eth.requestAccounts();
   await contract.methods.increment().send({ from:
accounts[0] });
}
async function getCount() {
   const result = await contract.methods.count().call();
   console.log(result);
}
```

4. Building and Deploying a Simple DApp on Ethereum Testnet

8. Connect MetaMask to Frontend

- Add button in HTML:

```
<button onclick="increment()">Increment</button>
```

- Ensure MetaMask:
  - Is installed
  - Connected to same testnet
  - Has test ETH

9. Test the DApp

✔ Click buttons

✔ Confirm transactions in MetaMask

✔ Check updated value

You can also verify on: Etherscan (use Sepolia version)

10. Deploy Frontend

Options:

- GitHub Pages
- Netlify
- Vercel

5. Integrating Cryptocurrency Payments in DApps

Cryptocurrency payments in DApps allow users to:

- Send/receive tokens (ETH, ERC-20)
- Pay for services (fees, subscriptions, NFTs)
- Interact with smart contracts using wallets

This is typically done using:

- MetaMask (wallet)
- Web3.js or Ethers.js (frontend interaction)

Services offered by Payment Gateways :

- Accept multiple cryptocurrencies
- Convert crypto → fiat
- Simplify UX

Types of Crypto Payments

a) Native Cryptocurrency Payments

- Paid in ETH (or testnet ETH)
- Ex: Paying gas or sending ETH

b) Token-Based Payments (ERC-20)

- Payments using tokens (USDT, DAI, custom tokens)

c) Smart Contract-Based Payments

- Payments handled via contract logic (escrow, subscriptions)

5. Integrating Cryptocurrency Payments in DApps

Basic Payment Flow Steps

1. Connect Wallet: User connects to the DApp via MetaMask or another provider.
2. Initiate Transaction: User clicks "Pay/Mint" in the UI, prompting the DApp to build a transaction.
3. Sign Request: The wallet pops up, showing transaction details (amount, recipient, gas fee) for user approval.
4. Submit & Confirm: The signed transaction is sent to the blockchain network.
5. Smart Contract Execution: The contract executes the logic and updates its state.
6. Confirmation: The DApp receives the transaction receipt and updates the UI for the user.

Courtesy : ChainLink

5. Integrating Cryptocurrency Payments in DApps

1. Smart Contract for Payments

Sample code to accept ETH Payments

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Payment {
   address public owner;
   constructor() {
       owner = msg.sender;
   }
   // Accept ETH
   function pay() public payable {}
   // Check balance
   function getBalance() public view returns(uint) {
       return address(this).balance;
   }
   // Withdraw funds
   function withdraw() public {
       require(msg.sender == owner, "Not owner");
       payable(owner).transfer(address(this).balance);
   }
}
```

5. Integrating Cryptocurrency Payments in DApps

2. Frontend Integration Using Web3.js:

Connect Wallet

```
await window.ethereum.request({ method: 'eth_requestAccounts' });
```

Send ETH Payment

```
const web3 = new Web3(window.ethereum);
async function sendPayment() {
   const accounts = await web3.eth.getAccounts();
   await web3.eth.sendTransaction({
       from: accounts[0],
       to: "CONTRACT_ADDRESS",
       value: web3.utils.toWei("0.01", "ether")
   });
}
```

5. Integrating Cryptocurrency Payments in DApps

3. ERC-20 Token Payment Integration

Step 1: Token Contract Interface

- Use ABI of ERC-20 token

Step 2: Approve + Transfer

```
const tokenContract = new web3.eth.Contract(abi, tokenAddress);
// Approve contract to spend tokens
await tokenContract.methods.approve(contractAddress, amount).send({ from: user });
// Call payment function
await contract.methods.payWithToken(amount).send({ from: user });
```

5. Integrating Cryptocurrency Payments in DApps

Security Considerations

- Validate sender (msg.sender)
- Use require() for checks
- Prevent reentrancy attacks
- Use OpenZeppelin contracts
- Avoid hardcoding sensitive data

Gas Optimization

- Minimize storage usage
- Use events instead of storing data
- Batch transactions when possible

Testing on Testnet

- Use testnets like Sepolia
- Get ETH from faucets
- Verify transactions on Etherscan

Payment Gateways (Advanced) for crypto payment services:

- Coinbase Commerce
- BitPay
- NOWPayments

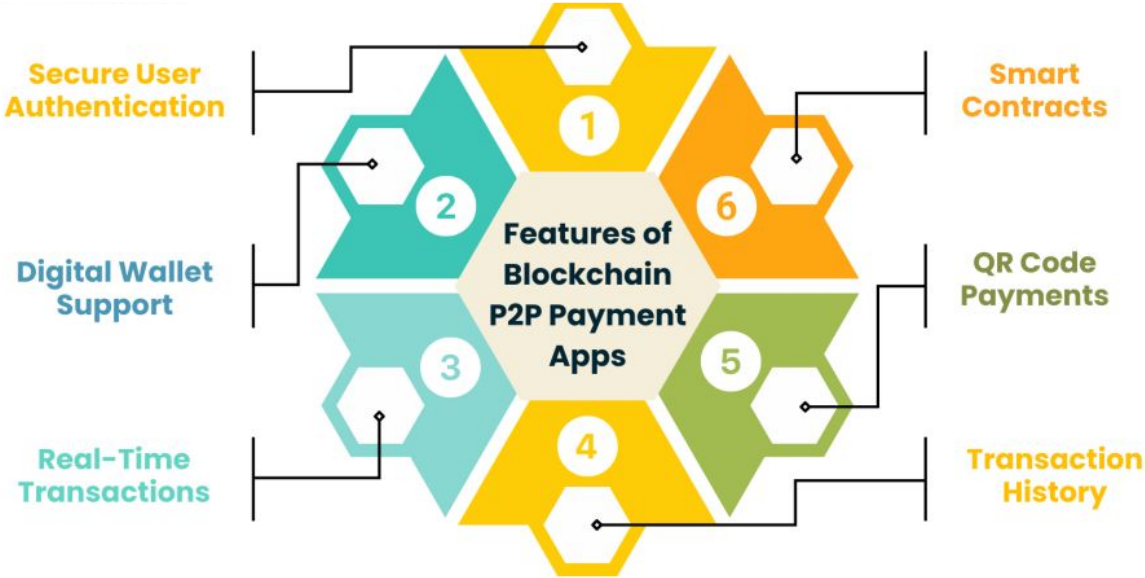
Real Use Cases

- NFT marketplaces (buy NFTs)
- DeFi apps (staking, lending)
- Subscription services
- Gaming (in-game purchases)

5. Integrating Cryptocurrency Payments in DApps

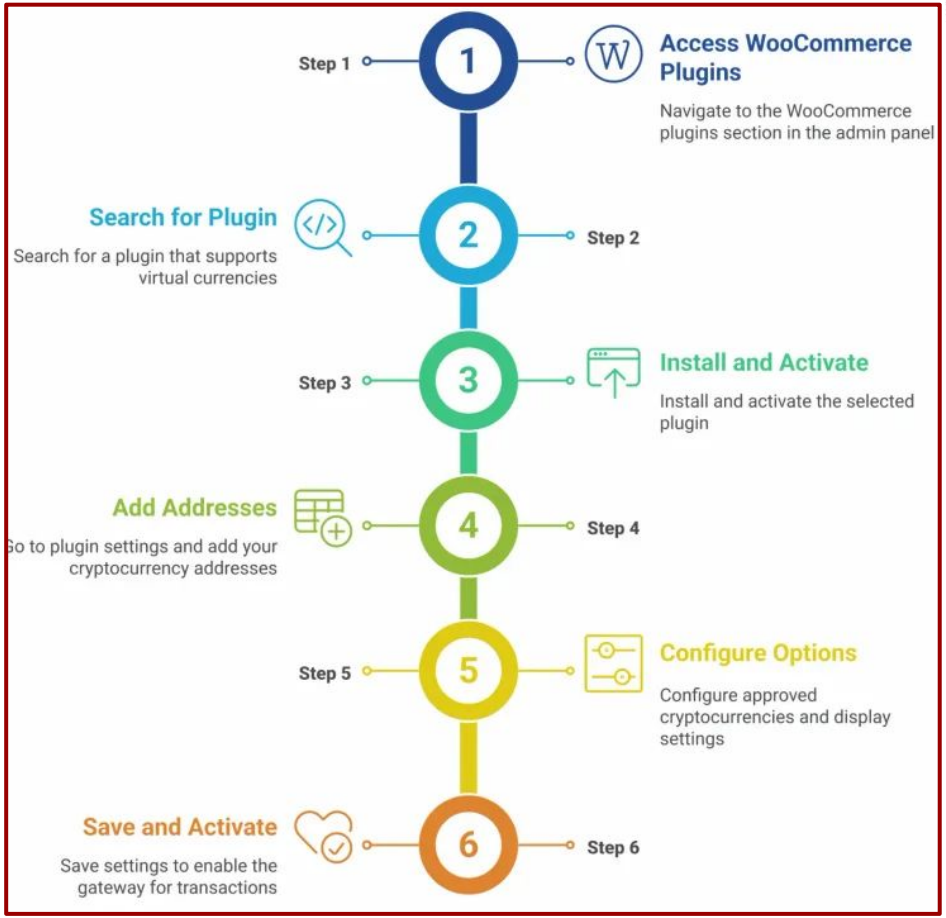
Steps to integrate crypto payments in a DApp:

1. Create payment-enabled smart contract
2. Deploy on testne
3. Connect frontend using Web3/Ethers
4. Enable wallet interaction
5. Handle ETH or token payments
6. Ensure security & test thoroughly



5. Integrating Cryptocurrency Payments in DApps

Setting up a Cryptocurrecny Payment Gateway using wooCommerce



6. Performance Optimization and Scaling Solutions for DApps

Why Optimization is Needed ?

DApps face:

- High gas fees
- Low throughput (~15–30 TPS on Ethereum L1)
- Network congestion
- Slow transaction confirmations

These issues affect **user experience, cost, and adoption**

Real-World Examples

- DeFi apps use Layer 2 (Arbitrum/Optimism)
- NFT platforms use Polygon for cheaper minting
- Games use state channels + sidechains

Best Practices

✔ Use Layer 2 by default

✔ Minimize on-chain storage

✔ Batch transactions

✔ Use efficient data structures

✔ Cache frequently used data

✔ Monitor gas usage

Tools for Optimization

- Hardhat (testing & profiling)
- Truffle
- Ganache
- Tenderly

6. Performance Optimization Techniques

Smart Contract Optimization

1. Efficient Data Storage
   - Use **memory** instead of storage
   - Pack variables to reduce storage slots
2. Reduce Gas Consumption
   - Avoid loops over large arrays
   - Use uint256 instead of smaller types (EVM optimized)
3. Use Events Instead of Storage
   - Store less on-chain
   - Emit events for logs
4. Reuse Code : Use libraries like OpenZeppelin

Frontend Optimization

- Minimize blockchain calls
- Cache data locally
- Batch requests
- Use: Ethers.js / Web3.js

Off-Chain Computation : Move heavy logic off-chain:

- Use backend servers or oracles
- Only store final results on-chain
- compute off-chain → store result

Efficient Data Indexing

- Use indexing services: The Graph
- Benefits: Faster queries & Reduced direct blockchain reads

6. Scaling Solutions for DApps

Layer 2 Scaling Solutions : Rollups (Most Popular)

- Optimistic Rollups : Assume transactions are valid
  - Ex: Optimism, Arbitrum
- ZK-Rollups : Use cryptographic proofs
  - Ex: zkSync, StarkNet
- Benefits:
  - Lower gas fees
  - Higher throughput

Sidechains

- Independent chains connected to Ethereum
- Ex: Polygon
- Pros: Fast transactions
- Cons: Less decentralization

State Channels

- Off-chain transactions between parties
- Only final state recorded on-chain
- Ex : Micropayments, Gaming

Sharding (Future Ethereum Scaling)

- Splits blockchain into smaller parts (shards)
- Parallel transaction processing
- Associated with: Ethereum 2.0

Cross-Chain Solutions

- Enable interoperability between blockchains
- Ex: Bridges, Polkadot ecosystem

7. Testing and Deploying DApps to Production Environments

Why Testing & Deployment Matter for DApps?

DApps are:

- Immutable once deployed
- Handle real money/assets
- Publicly accessible

Bugs can lead to **financial loss or exploits, so rigorous testing is essential.**

7. Types of Testing in DApps

1. Unit Testing (Smart Contracts)

- Test individual functions
- Validate logic (e.g., transfers, balances)
- Tools: Hardhat, Truffle

Example (Hardhat test):

```
describe("Counter", function () {
 it("Should increment count", async function () {
   const Counter = await ethers.getContractFactory("Counter");
   const counter = await Counter.deploy();

   await counter.increment();
   expect(await counter.count()).to.equal(1);
 });
});
```

2. Integration Testing

- Test interaction between:
  - Smart contracts
  - Frontend
  - Wallets
- Use: Ethers.js , Web3.js

3. Frontend Testing

- UI/UX testing
- Wallet interaction testing
- Tools: Jest, Cypress

7. Types of Testing in DApps

4. Security Testing

- Key Checks:
  - Reentrancy attacks
  - Integer overflow/underflow
  - Access control issues
- Use: OpenZeppelin / Slither / MythX

5. Gas Testing

- Optimize transaction costs
- Tools: Hardhat Gas Reporter / Tenderly

Testing on Local Blockchain

- Use: Ganache
- Steps:
  1. Deploy contracts locally
  2. Simulate transactions
  3. Debug errors

Courtesy : Himanshi Blog

7. Testnet Deployment

- Before production, deploy on testnets:
- Popular Testnets: Sepolia / Goerli (deprecated in many cases)
- Steps:
  1. Configure network in: Hardhat or Truffle
  2. Add test ETH via faucet
  3. Deploy using scripts

	Mainnet	Testnet
Purpose	Functional Blockchain	Testing Environment
Transactions	Real Transactions Stored on Actual Blockchain	Fake Transactions
Coins	Posses Real Value	No Monetary Value
Transaction Cost	Paid Using Native Coins	No Cost (Free)
Transaction Frequency	High	Low
Mining	Economic Incentive to Earn Real Coins	No Economic Benefit

Courtesy : phemex.com

7. Deployment Process (Step-by-Step)

Step 1: Prepare Smart Contract

- Audit code
- Optimize gas
- Remove debug logs

Step 2: Secure Private Keys

- Use environment variables (`.env`)
- Never hardcode keys

Step 3: Deploy to Mainnet

- Using Hardhat:

```
npx hardhat run scripts/deploy.js --network mainnet
```

Step 4: Verify Smart Contract

- Use: Etherscan
  - Makes contract transparent and trustworthy

Step 5: Deploy Frontend

- Platforms: Vercel / Netlify / GitHub Pages

Step 6: Connect Frontend to Mainnet

- Update:
  - Contract address
  - Network configuration

7. Testing and Deploying DApps to Production Environments

Best Practices for Production

✔ Conduct smart contract audits

✔ Use multi-signature wallets

✔ Implement upgradeable contracts

✔ Monitor transactions

✔ Enable logging & alerts

CI/CD for DApps

- Automate: Testing & Deployment
- Tools: GitHub Actions / GitLab CI/CD

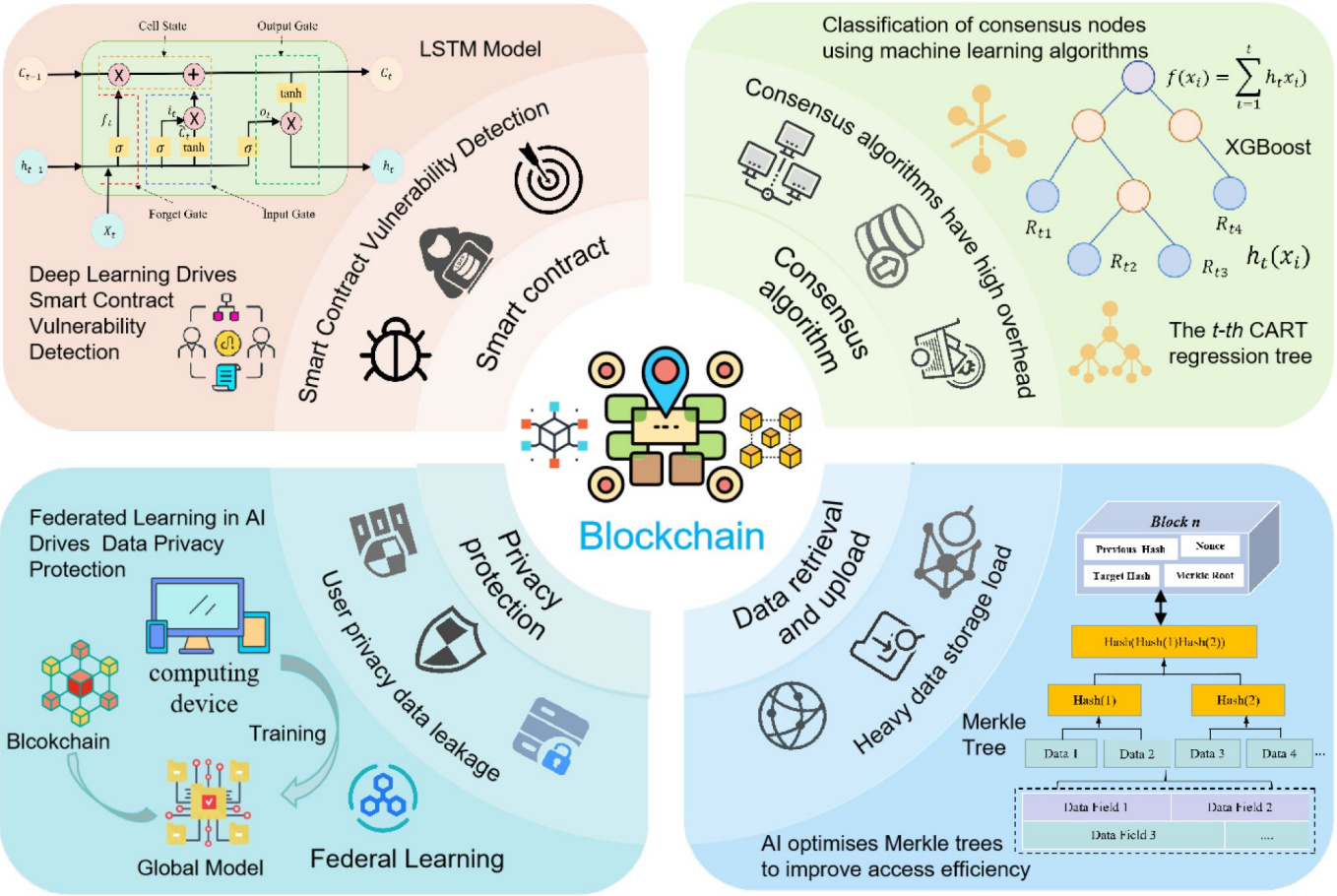
Monitoring & Maintenance

- Use: Tenderly / Alchemy / Infura

Common Deployment Mistakes

- Deploying without testing
- Hardcoding private keys
- Ignoring gas optimization
- Not verifying contract
- Poor access control

Existing challenges of AI-driven blockchain



Courtesy : MDPI-Paper